CLEANFLOW / GESTICOLLECTE

Architecture Détaillée des 5 Agents d'Intelligence Artificielle

---

Projet de Fin d'Études (PFE) — Plateforme Municipale de Gestion Intelligente des Déchets

*Ce document fournit une **cartographie exhaustive et technique** des 5 Agents Intelligents intégrés dans la plateforme CleanFlow. Chaque agent est documenté en profondeur avec son **rôle opérationnel**, ses **modèles LLM / algorithmes**, ses **spécifications d'API REST**, le **détail exact de ses fichiers sources Java/Spring Boot** et son **interface utilisateur** (Web React et Mobile Flutter).*

Vue Synoptique des 5 Agents Intelligents

Agent IA	Famille Technologique	Modèle / Moteur	Rôle Métier Principal
1. Assistant Chauffeur RAG	RAG & NLP Multilingue	Gemini Embedding-2 (768d) + Groq (Qwen 27B)	Support chauffeur en Derja tunisienne & français, procédures, pannes
2. Assistant Superviseur	Function Calling / Agentic	Groq / Qwen 27B (JSON Tools)	Pilotage opérationnel, recherche chauffeurs, alertes, bacs
3. Vision Contrôle Propreté	Computer Vision Multimodal	Groq / Qwen 27B Vision	Audit propreté AVANT/APRÈS vidage, détection débordements & anomalies
4. Recrutement & Filtrage CV	Information Extraction & Scoring	Groq / Qwen 27B (Strict Schema)	Extraction CV PDF/Word, scoring multicritère pondéré des chauffeurs
5. Optimisation des Tournées	Recherche Opérationnelle (TSP)	Nearest Neighbor, 2-Opt, OpenRouteService	Routage routier réel, minimisation carburant & temps de trajet

Agent 1 : Assistant Chauffeur RAG Multilingue (Derja Tunisienne & Français)

Mission opérationnelle : Assister le chauffeur en temps réel avant, pendant et après sa tournée de collecte. L'agent répond aux questions relatives aux consignes de sécurité, contrôle du véhicule, protocole de vidage des bacs, pannes mécaniques et gestion des réclamations.

Caractéristiques Techniques Clés :

- **Support natif de la Derja Tunisienne :** Compréhension et génération en Derja en alphabet latin (Arabizi : 3, 5, 7, 9) et en écriture arabe. Préservation des termes techniques métier tunisiens (tournée, bac, peigne, benne).
- **Recherche Hybride Petits Corpus :** Combinaison robuste de BM25 lexical et de similarité Cosine sur vecteurs d'embeddings sémantiques 768 dimensions (Google Gemini Embedding-2). Aucun cluster vectoriel externe requis, tout est géré nativement via MongoDB.
- **Garantie Zéro Hallucination & Validation Stricte des Citations :** Chaque consigne municipale générée doit obligatoirement être accompagnée d'une citation textuelle vérifiée dans les documents officiels. Normalisation intelligente des espaces et puces typographiques.
- **Résilience & Mode Dégradé :** En cas de quota dépassé ou panne fournisseur LLM (ex: HTTP 429), bascule automatique transparente vers l'extraction brute SOURCE_EXCERPTS.
- **Intégration Vocale Complète :** Bouton microphone Speech-to-Text (STT) et lecture vocale Text-to-Speech (TTS) dans l'application mobile Flutter.

Fichiers de Code Source — Backend & Mobile :

Fichier Source	Rôle & Responsabilité dans l'Agent
`ai/rag/service/RagService.java`	Cœur orchestrateur du pipeline RAG : buildPrompt(), exécution LLM, validation stricte des citations via isQuoteSupported(), gestion du repli SOURCE_EXCERPTS.
`ai/assistant/service/TunisianDialectService.java`	Détection automatique de la langue (Arabizi, Arabe, Français), injection des directives impératives de dialecte, messages de repli localisés.
`ai/rag/service/MongoVectorStoreService.java`	Recherche hybride (BM25 + Cosine similarity), tokenisation avec filtrage stop-words, normalisation NFD, calcul de classement combiné RRF.
`ai/rag/service/GeminiEmbeddingService.java`	Génération des vecteurs d'embedding 768 dimensions via l'API REST Google Gemini (gemini-embedding-2).
`ai/rag/service/DocumentProcessingService.java`	Pipeline d'ingestion asynchrone des documents PDF/Word : extraction de texte, découpage en chunks, calcul d'embeddings, stockage MongoDB.
`ai/rag/service/ChunkingService.java`	Algorithme de découpage textuel intelligent en morceaux de 1000 caractères avec chevauchement (overlap) de 150 caractères.
`ai/rag/service/DocumentLoaderService.java`	Extraction de texte robuste pour fichiers PDF (Apache PDFBox), DOCX (Apache POI), TXT et MD.
`ai/rag/controller/RagController.java`	Endpoint REST POST /api/ai/rag/query exposé pour les clients web et mobile (authentification JWT Bearer).
`ai/rag/controller/DocumentController.java`	Endpoints de téléversement, approbation superviseur et réindexation vectorielle (POST /api/ai/documents/{id}/reindex).
`ai/rag/model/AiDocument.java & DocumentChunk.java`	Entités MongoDB stockant les métadonnées documentaires et les chunks de texte avec leurs vecteurs de flottants.
`MOBILE/lib/pages/assistant_page.dart`	Interface de chat mobile Flutter : gestion des bulles, saisie vocale (speech_to_text: ^7.4.0), synthèse vocale (flutter_tts: ^4.2.5).

Agent 2 : Assistant Superviseur Opérationnel (Function Calling / Tool Use)

Mission opérationnelle : Offrir au superviseur municipal un copilote interactif capable de répondre à des requêtes en langage naturel sur l'état de la flotte, les retards de collecte, les chauffeurs d'astreinte, et de déclencher des requêtes dynamiques en base de données.

Fonctionnement par Exécution d'Outils (Tools) :

Contrairement à un simple chatbot conversationnel, cet agent analyse l'intention de l'utilisateur et sélectionne automatiquement un outil Java à exécuter :

1. consulterPointsCollecte(statut, zone) : Récupère la liste des bacs nécessitant une intervention urgente (pleins ou débordants).
2. rechercherChauffeursDisponibles(date) : Identifie les chauffeurs actifs n'ayant pas de tournée en cours.
3. consulterAlertesEtReclamations(urgence) : Synthétise les signalements terrain (bacs inaccessibles, pannes).
4. verifierEtatFlotteCamions() : État des bennes tasseuses (en service, en panne, en pesée).

Fichiers de Code Source :

Fichier Source	Rôle & Responsabilité dans l'Agent
`ai/assistant/service/AssistantService.java`	Gestion du cycle conversationnel superviseur, historique des échanges, orchestration de l'appel au modèle Groq Qwen 27B.
`ai/assistant/service/tools/AssistantTools.java`	Catalogue des outils disponibles : définition des schémas JSON des fonctions, injection des repositories Spring Data et exécution des requêtes métiers.
`ai/assistant/service/AssistantIntent.java`	Classificateur déterministe d'intentions : distingue salutations, demandes de rédaction de rapports municipaux, et requêtes documentaires.
`ai/assistant/controller/AssistantController.java`	Endpoint REST POST /api/ai/assistant/chat sécurisé par le rôle SUPERVISEUR.
`ai/assistant/model/ChatConversation.java & ChatMessage.java`	Historique persistant des conversations d'assistance superviseur dans MongoDB.

Agent 3 : Agent Vision par Ordinateur & Contrôle Qualité de Collecte

Mission opérationnelle : Automatiser la certification de la qualité de la collecte des déchets en analysant les photographies prises sur le terrain par les chauffeurs via leur smartphone (photos AVANT et APRÈS le vidage du conteneur).

Workflow d'Évaluation Multimodale :

- **Entrées :** 1 ou 2 photographies compressées (max 10 Mo par photo, format JPEG/PNG/WebP) et métadonnées du point de collecte.
- **Moteur Multimodal :** LLM Vision haute performance hébergé sur Groq (qwen/qwen3.8-27b) avec prompt structuré et schéma JSON strict.
- **Classification des Statuts de Propreté :**

- PROPRE : Point de collecte vidé, sol nettoyé, bac remis en place sans débordement résiduel.
- DEBORDEMENT : Sacs poubelles au sol ou volume résiduel non ramassé.
- NON_CONFORME : Présence de déchets interdits (gravats, béton, ferraille, produits toxiques).
- INACCESSIBLE : Véhicule mal garé ou travaux bloquant le conteneur.
- CANNOT_DETERMINE : Photo floue, cadrage insuffisant ou prise de nuit sans éclairage.

- **Validation Métier :** L'analyse génère une justification textuelle factuelle en français, stockée et liée à la réclamation ou au point validé.

Fichiers de Code Source :

Fichier Source	Rôle & Responsabilité dans l'Agent
`ai/vision/service/ImageAnalysisService.java`	Validation des fichiers images, encodage Base64, construction du prompt d'inspection, appel au client vision Groq, persistance de l'analyse.
`ai/vision/controller/VisionController.java`	Endpoints REST POST /api/ai/vision/analyze (multipart avec 1 ou 2 images) et POST /api/ai/vision/evaluate.
`ai/vision/model/ImageAnalysis.java`	Entité MongoDB enregistrant l'URL des photos, le statut détecté, la description explicative, l'identifiant du point de collecte et la date.
`ai/vision/model/AiProblem.java`	Énumération des anomalies terrain formalisées (DEBORDEMENT, BAC_CASSE, NON_CONFORME, ACCES).
`ai/common/GroqClient.java`	Méthode analyzeImages() : gestion des requêtes multimodales OpenAI-compatible vers l'infrastructure Groq avec schéma JSON strict.
`pointcollecte/PointValidationTest.java`	Tests d'intégration validant le passage automatique du point de collecte à l'état VALIDÉ suite à l'analyse vision réussie.

Agent 4 : Agent Recrutement & Scoring Intelligent des Chauffeurs (CV Screening)

Mission opérationnelle : Assister le service des ressources humaines de la municipalité dans le tri, l'extraction structurée et l'évaluation multicritère des candidatures de chauffeurs pour bennes tasseuses.

Architecture & Pipeline d'Évaluation :

1. **Extraction Structurée Sans Hallucination :** Le document (PDF/Word/Texte) est analysé par le LLM qui extrait en JSON strict les informations clés : permis de conduire (B, C, CE), années d'expérience en collecte, habilitations sécurité (FIMO/FCO), zone de résidence, etc.

2. **Exigences Municipales Configurables (DriverRequirements) :** Le superviseur RH peut ajuster les critères d'embauche et leurs coefficients de pondération (ex: Permis C obligatoire avec poids 40, expérience benne tasseuse poids 30, proximité géographique poids 15).

3. **Traçabilité des Preuves (FieldEvidence) :** Chaque note attribuée est obligatoirement justifiée par un extrait textuel littéral (snippet) extrait du CV original du candidat.

4. **Scoring Global & Décision :** Calcul d'une note pondérée sur 100 avec mention d'admissibilité (ADMISSIBLE, ATTENTION, REJETÉ).

Fichiers de Code Source :

Fichier Source	Rôle & Responsabilité dans l'Agent
`ai/cv/service/CvAnalysisService.java`	Chef d'orchestre de l'analyse CV : chargement du fichier, coordination extraction, scoring et sauvegarde des candidats.
`ai/cv/service/CvExtractionService.java`	Extraction structurée des champs du CV via prompt avec schéma JSON rigoureux interdisant l'extrapolation.
`ai/cv/service/CandidateScoringService.java`	Moteur de calcul mathématique des scores : évaluation de chaque critère selon les exigences municipales courantes.
`ai/cv/service/DriverRequirementsService.java`	Gestion du référentiel des exigences de recrutement (seuils minimaux, pondérations, versionnage).
`ai/cv/controller/CvController.java`	Endpoints REST : POST /api/ai/cv/upload, GET /api/ai/cv/candidates, PUT /api/ai/cv/requirements.
`ai/cv/model/Candidate.java`	Entité MongoDB du candidat : données personnelles, permis, expérience, score global et statut de recrutement.
`ai/cv/model/CandidateEvaluation.java & CriterionScore.java`	Détail du scoring critère par critère avec note, note pondérée et extrait du CV justifiant la décision.
`ai/cv/model/DriverRequirements.java`	Définition des critères d'évaluation municipaux configurables par l'administrateur.

Agent 5 : Agent d'Optimisation des Tournées & Routage Routier (Smart Routing)

Mission opérationnelle : Calculer l'ordre de passage optimal sur l'ensemble des points de collecte d'une tournée pour minimiser la distance totale parcourue, le temps de service et la consommation de carburant de la benne tasseuse, en partant et revenant au dépôt municipal.

Algorithmes Mathématiques et Routage Réel :

- **Problème du Voyageur de Commerce (TSP) avec Dépôt :** Résolution NP-difficile adaptée à la collecte municipale avec contrainte de départ et retour au dépôt officiel (ex: dépôt municipal Ben Arous / Jasmin).
- **Algorithme Heuristique du Plus Proche Voisin (Nearest Neighbor) :** Construction rapide d'un cycle initial valide en sélectionnant séquentiellement le point le plus proche non encore visité.
- **Métaheuristique 2-Opt Local Search :** Amélioration itérative du cycle par décroisement des arêtes. Réduit drastiquement les allers-retours inutiles et évite les manœuvres dangereuses de marche arrière pour les poids lourds.
- **Calcul de Matrice de Distance Routière Réelle (OpenRouteService) :** Prise en compte du réseau routier réel, des sens uniques, des restrictions poids lourds et du profil de conduite routière (driving-car).
- **Fallback Géodésique (Formule de Haversine) :** En cas d'indisponibilité du serveur de routage externe, calcul instantané de la distance orthodromique à vol d'oiseau garantissant la continuité de service.

Fichiers de Code Source :

Fichier Source	Rôle & Responsabilité dans l'Agent
`tournee/service/RouteAlgorithms.java`	Implémentation pure en Java des algorithmes mathématiques : calculateDistanceMatrix(), nearestNeighborRoute(), twoOptOptimization(), haversine().
`tournee/service/OpenRouteServiceClient.java`	Client HTTP OkHttp communicant avec l'API OpenRouteService pour extraire la matrice de distances et durées réelles.
`tournee/service/TourneeRouteOptimizationService.java`	Application de l'optimisation aux tournées municipales réelles : extraction des coordonnées des bacs, exécution du routage, mise à jour de l'ordre des arrêts.
`tournee/service/RouteOptimizationResult.java`	Objet DTO encapsulant le résultat de l'optimisation : ordre ordonné des points, distance totale (km), durée estimée (min) et gain pourcentage.
`ai/route/service/RouteOptimizationService.java`	Service passerelle pour les simulations et évaluations expérimentales d'itinéraires.
`ai/route/controller/RouteOptimizationController.java`	Endpoint REST POST /api/ai/route/optimize pour tester des parcours arbitraires de points GPS.
`tournee/service/RoutingBenchmarkTest.java`	Suite de tests de charge et benchmarks comparant Nearest Neighbor vs 2-Opt avec export des résultats en CSV.

Couche Infrastructure Commune & Configuration Centralisée

Tous les agents IA de CleanFlow s'appuient sur une **couche d'infrastructure commune et agnostique** permettant de basculer facilement d'un fournisseur d'IA à un autre sans modifier le code métier de l'application.

Architecture Multi-Fournisseurs (Multi-Provider Strategy) :

- **Groq (Fournisseur Principal Production) :** Utilisé pour la génération de texte ultra-rapide (LPUs Groq) sur le modèle qwen/qwen3.8-27b et pour l'analyse d'images de vision. Très faible latence (< 1 seconde).
- **Google Gemini (Embeddings & Fallback) :** Fournisseur officiel des représentations vectorielles sémantiques 768 dimensions via le modèle gemini-embedding-2.
- **OpenRouter (Fournisseur Alternatif Gratuit) :** Configurable via .env en alternative pour l'analyse de CV et les photos de vision avec qwen/qwen3.8-27b:free.

Fichiers de Code de l'Infrastructure Commune :

Fichier Source	Rôle dans l'Architecture IA
`ai/config/AiProperties.java`	Configuration typée Spring Boot (@ConfigurationProperties(prefix='ai')) : clés d'API, modèles, seuils de similarité, timeouts et répertoires de stockage.
`ai/config/AiConfig.java`	Configuration des beans Spring : initialisation des clients HTTP OkHttp optimisés, pools de connexions et mappers Jackson ObjectMapper.
`ai/common/GroqClient.java`	Client HTTP pour l'API Groq : gestion des requêtes chat completion, fallback de format json_schema vers json_object et gestion des rate limits.
`ai/common/GeminiClient.java`	Client HTTP pour l'API Google Gemini : envoi des prompts texte et gestion des tokens.
`ai/common/OpenRouterClient.java`	Client HTTP pour l'API OpenRouter OpenAI-compatible pour les modèles hébergés gratuits.
`ai/common/AiServiceException.java`	Gestionnaire d'exceptions unifié pour tous les agents : catégorisation des erreurs (NOT_CONFIGURED, PROVIDER_HTTP_ERROR, INVALID_RESPONSE) sans fuite de secrets.
`ai/common/RagGenerationClient.java`	Interface d'abstraction commune implémentée par GeminiClient et GroqClient pour la génération de texte RAG.
`BACK/.env`	Fichier de variables d'environnement locales privées (ignoré par Git) contenant les clés GEMINI_API_KEY, GROQ_API_KEY, ORS_API_KEY et les modèles actifs.

Synthèse pour la Soutenance du PFE

Points forts à valoriser devant le jury :
*1. **Complétude opérationnelle :** Les 5 agents couvrent l'intégralité du cycle de vie de la gestion municipale des déchets (Ressources Humaines, Planification de tournées, Exécution terrain chauffeur, Contrôle qualité citoyen/superviseur).*
*2. **Souveraineté des données & Absence de bases externes payantes :** Le moteur vectoriel RAG est entièrement auto-hébergé sur MongoDB (BM25 + Cosine), évitant la dépendance et les coûts d'une base vectorielle cloud externe (comme Pinecone ou Qdrant).*
*3. **Inclusion et accessibilité linguistique :** Premier assistant municipal tunisien supportant nativement le dialecte Derja (Arabizi) et le vocal dans une application mobile Flutter de terrain.*
*4. **Fiabilité académique :** 256 tests unitaires et d'intégration validant le non-déterminisme, l'absence d'hallucinations et la robustesse en cas de panne réseau.*